A Microservices-Based Home Energy Tracker for Real-Time Monitoring, Intelligent Alerting, and AI-Driven Energy Insights

Rohit Shrivastava[1], Anshika Rai[2], Harsh Gupta[3], Syamasudha Veeragandham[4]
[1,2,3,4]School of Computer Science and Engineering, Vellore Institute of Technology, Vellore, Tamil Nadu, India – 632014

Email: [1]rohit.lalbahadur2023@vitstudent.ac.in, [2]anshika.rai2023@vitstudent.ac.in, [3]harsh.gupta2023@vitstudent.ac.in, [4] syamasudha.v@vit.ac.in

[4]Corresponding author

Abstract - Residential energy-monitoring platforms require a traceable route from device telemetry to actionable feedback, yet sensing, analytics, and scalable software deployment are often treated separately. This work presents a microservices-based Home Energy Tracker that integrates Wokwi-generated ESP32 telemetry, REST ingestion, Apache Kafka event transfer, InfluxDB time-series storage, MySQL metadata, deterministic energy analytics, and threshold-based alerting. The prototype comprises User, Device, Ingestion, Usage, Insight, and Alert services behind an API Gateway. The Insight Service calculates a slab-based bill forecast, appliance ranking, and a standby-load heuristic; an optional Ollama component supplies narrative recommendation wording only. In a controlled local functional verification, four synthetic telemetry events were accepted through the API path. Three analytics events totalled 1.50 kWh and produced deterministic forecast, ranking, and standby outputs, while one 1.00 kWh event exceeded a configured 0.10 kWh threshold and produced a persisted alert and Mailpit-captured email. Analytics and alert outputs were observed after 17.32 s and 6.14 s, respectively; both are polling-bounded functional observations rather than latency benchmarks. The contribution is a traceable integration baseline, not a calibrated meter, validated predictive model, scalability study, or resilience evaluation.

Keywords - *Home Energy Tracker, Internet of Things, microservices, Spring Boot, Apache Kafka, InfluxDB, Wokwi simulation, threshold alerting.*

I. INTRODUCTION

Residential electricity bills provide limited visibility into when energy is consumed, which registered device contributes most, or when a configured consumption threshold has been exceeded. A practical Home Energy Management System therefore requires an auditable path from telemetry to stored records, interpretable outputs, and user notification. Recent IoT monitoring systems demonstrate affordable sensing and remote visualisation, whereas other studies focus on energy analytics or anomaly detection [3]-[6], [13], [16], [18]-[21], [23].

This work addresses the system-integration problem rather than the accuracy of an energy meter or a trained analytical model. The Home Energy Tracker separates household and device metadata, telemetry ingestion, time-series usage, deterministic insights, and alert handling into independently deployable Spring Boot services. Wokwi supplies timestamped synthetic JSON payloads to the same REST interface intended for a future meter. Kafka decouples ingestion from downstream storage, InfluxDB stores the usage series, and Mailpit captures local alert-delivery evidence.

The study makes three bounded contributions: it implements an event-driven microservices path from simulated telemetry to time-series storage; it provides reproducible, deterministic billing, ranking, standby, and threshold calculations; and it documents controlled end-to-end evidence for ingestion, persistence, analytics, alert creation, and local email capture. The scope excludes calibrated sensing, supervised anomaly detection, forecast validation against a physical meter or utility bill, load testing, resilience testing, and demonstrated authentication or authorization.

II. RELATED WORK AND RESEARCH GAP

A. Existing Approaches

The literature represented in the source paper spans three complementary directions. Low-cost IoT systems use ESP32-class controllers and energy sensors to expose consumption or power-quality information [3]-[6], [13], [16], [18]-[21], [23]. These systems motivate a future calibrated sensing phase, but their measurement evidence cannot be transferred to the Wokwi-generated values used here. A second direction uses machine-learning or statistical methods for appliance classification, anomaly detection, and behavioural analysis [1], [7]-[11], [15], [22]. Such studies show the value of labelled time-series data, but the present prototype does not implement a trained classifier or report classification metrics.

A third direction considers microservices, cloud organisation, or distributed control for energy-management systems [2], [12], [14], [17], [24]. These works motivate service separation and maintainability, while leaving an opportunity to demonstrate a concise, traceable prototype that links a telemetry contract, an event broker, time-series persistence, deterministic user-facing insights, and a notification path. The present work adopts this integration focus without claiming the scalability or fault tolerance described as design goals in prior architectural research.

B. Research Gap and Contribution

The principal gap is the limited implementation-level linkage between residential telemetry, event-driven services, historical time-series storage, transparent analytics, and alert delivery. The prototype addresses this gap at the integration level through Wokwi telemetry, API Gateway routing, Spring Boot microservices, Kafka, InfluxDB, MySQL, deterministic insight functions, and Mailpit-backed threshold alerting. It does not resolve the related gaps of benchmark datasets, labelled anomaly data, contextual energy analysis, resilience validation, or privacy-preserving access control. These unresolved gaps are retained as limitations rather than treated as completed claims.

III. PROPOSED SYSTEM AND METHODOLOGY

A. Architecture and Telemetry Path

The input record contains deviceId, energyConsumed, and timestamp. In the Wokwi firmware, a virtual ESP32 creates this JSON payload at an approximately 8 s interval. Eco, Heavy, and Alert Test modes generate controlled values for integration testing; they are not calibrated physical measurements. The API Gateway routes the request to the Ingestion Service, which requires deviceId and timestamp and rejects negative energyConsumed values. Accepted records are mapped to energy-usage events and published to Kafka.

The Usage Service consumes the event and writes a time-stamped InfluxDB energy_usage point with deviceId as a tag and energyConsumed as a field. User and Device Services maintain household, appliance, and alert-setting metadata in MySQL. Aggregation over a selected D-day window produces device and household totals. This separation lets the telemetry-facing request be acknowledged without waiting for analytical or notification work.

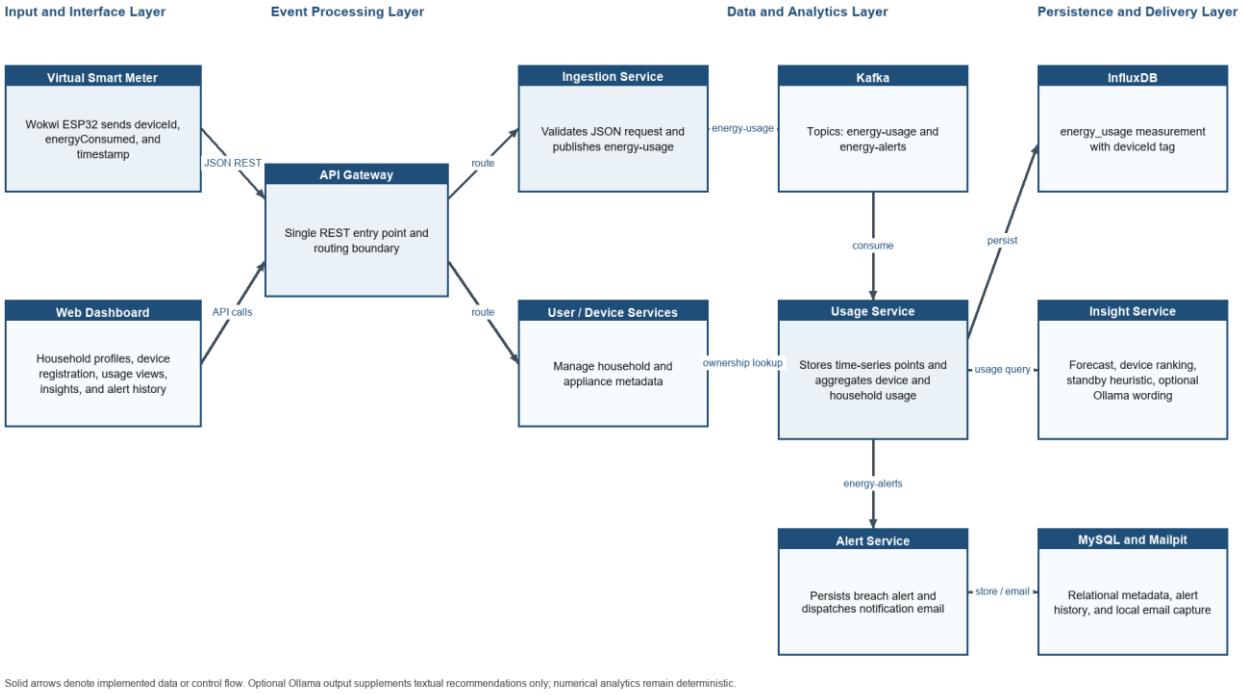


Fig. 1. Proposed architecture of the Home Energy Tracker.

B. Deterministic Insight and Alert Methods

The Insight Service transforms aggregated usage rather than learning features from a training set. It derives a daily rate and a 30-day energy projection from total energy E over D selected days. The configured local tariff applies a fixed 10.00 charge and slab rates of 0.12 per kWh for 0-100 kWh, 0.18 per kWh for 100-300 kWh, and 0.25 per kWh above 300 kWh. These are demonstration parameters used to verify application logic, not a utility-approved tariff.

$$E_{day} = \frac{E_{tot}}{D}; \quad E_{30} = 30\ x\ E_{day}$$

Devices are ordered by descending energyConsumed and each contribution is calculated as the device total divided by the household total. The Standby Energy Detective estimates baseline demand as 20% of average load. It labels the estimate LOW at or below 50 W, MEDIUM above 50 W and at or below 150 W, and HIGH above 150 W. This is an explainable heuristic and does not identify actual low-activity periods or appliance-level standby demand.

$$P_{avg} = (\frac{E_{tot}}{24D})\ x\ 1000\ W; \quad P_{base} = 0.20\ x\ P_{avg}$$

For alerting, a scheduler evaluates the preceding one-hour household sum every 10 s. When alerting is enabled and the sum exceeds the user-defined threshold, an energy-alerts event is published. The Alert Service persists the alert in MySQL and attempts notification delivery through the configured SMTP endpoint, with Mailpit used as the local demonstration inbox. The current rule does not deduplicate repeated threshold breaches.

TABLE I

Implemented System Components and Traceable Data Flow

Stage	Implemented component and technique	Output
Telemetry and routing	Wokwi ESP32 virtual meter sends deviceId, energyConsumed, and timestamp through the API Gateway.	REST request delivered to the Ingestion Service.
Ingestion and event transfer	Bean validation checks mandatory fields and non-negative energy; accepted payloads become Kafka energy-usage events.	Asynchronous energy-usage event.
Storage and aggregation	Usage Service writes energy_usage points to InfluxDB and sums selected time windows; Device Service resolves ownership.	Device and household energy totals.

Stage	Implemented component and technique	Output
Insight computation	Slab forecast, device ranking, and 20% average-load standby heuristic use aggregated values.	Deterministic numerical outputs and recommendations.
Threshold alerting	A 10 s scheduler compares the preceding one-hour total with the configured threshold; Alert Service persists and sends the event.	Alert history and Mailpit-captured notification.

C. Operational Workflow

Fig. 2 summarises the operational sequence. The validation, event-storage, insight, and alert branches are intentionally separated. Functional evaluation records the HTTP response, queryable usage, deterministic insight response, alert history, and notification evidence. No supervised learning, train-test split, or empirical anomaly-classifier evaluation is present in the current prototype.

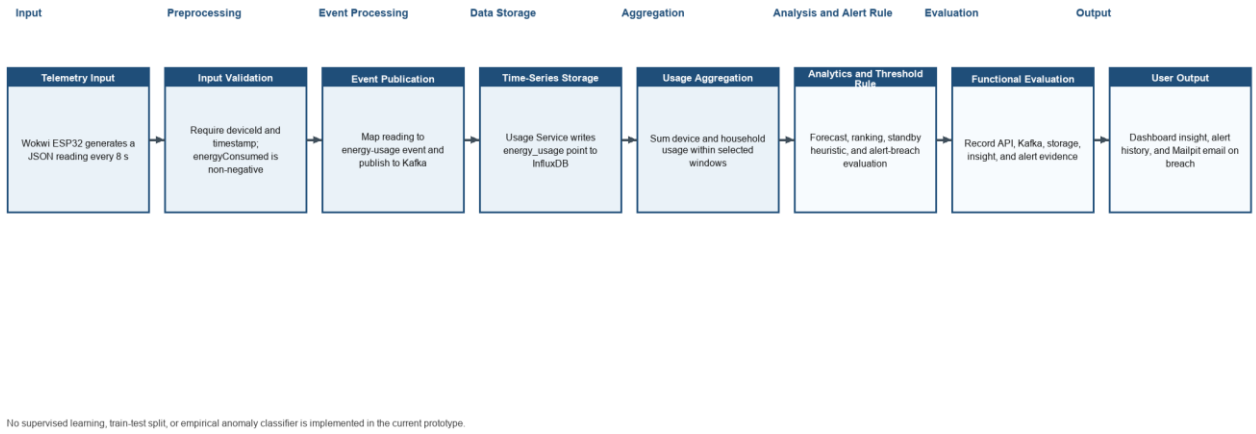


Fig. 2. Functional workflow of the proposed Home Energy Tracker.

IV. IMPLEMENTATION AND FUNCTIONAL VERIFICATION

A. Implementation Environment

The prototype was executed as a local Docker-based environment on Windows 11 Home Single Language (10.0.26200) with an 11th Gen Intel Core i5-1135G7 processor at 2.40 GHz, four cores, eight logical processors, and 7.74 GB RAM. The C: drive had 373.71 GB total capacity and 98.13 GB free during verification. The deployment included User, Device, Ingestion, Usage, Insight, Alert, and API Gateway services, together with the frontend, Kafka, MySQL, InfluxDB, and Mailpit containers.

TABLE II

Controlled Verification Environment and Conditions

Element	Documented configuration
Runtime and deployment	Docker Engine 29.7.2 and Docker Compose v5.5.1. Java 17; Spring Boot 3.5.3 for User, Device, Ingestion, Usage, Alert, and Insight services; API Gateway uses Spring Boot 3.4.5.
Application and data stack	React 19.2.8, Vite 8.2.2, TypeScript 6.0.2, Tailwind CSS 4.3.3, Apache Kafka 4.3.1 observed from the running container, MySQL 8.3.0, and InfluxDB 2.7.
IoT simulation	ESP32 DevKit, potentiometer, three pushbuttons, and SSD1306 OLED simulated in Wokwi. PlatformIO IDE VS Code extension 3.3.4 and Wokwi VS Code extension 3.7.0 were used.
Controlled data	Four valid synthetic JSON events on 1 October 2026: 0.25, 0.50, and 0.75 kWh for analytics device 307, and 1.00 kWh for alert device 308. Fields: deviceId, energyConsumed, timestamp.
Insight configuration	Three-day insight window; 30-day forecast horizon; fixed 10.00 charge; 0.12, 0.18, and 0.25 per-kWh slab rates; 20% standby coefficient.
Alert configuration	One-hour aggregation window and 10 s scheduler. User 9 threshold set to 0.10 kWh for the isolated alert case. Spring AI 1.0.3 is present; Ollama CLI was not installed or used during controlled verification.

B. Verification Procedure

1. A fresh household and device were registered, and the analytics case submitted three timestamped REST telemetry records (0.25, 0.50, and 0.75 kWh) for device 307.

2. HTTP responses, usage totals, and Insight Service outputs were queried after ingestion. The selected insight window was D = 3 days.
3. An isolated alert case used user 9 with alerting enabled and a 0.10 kWh threshold. One 1.00 kWh event was submitted for device 308, then alert history and Mailpit were inspected.
4. The local Maven suite covered the stateful and analytical services; 19 tests were discovered, 16 executed with zero failures and zero errors, and three were marked skipped.

V. RESULTS AND DISCUSSION

A. Functional Results

The controlled run establishes functional traceability rather than general performance. All four submitted events returned HTTP 201. Device 307 produced a queryable 1.50 kWh total, while device 308 produced a queryable 1.00 kWh total in the isolated alert case. The records originated from controlled REST submissions matching the Wokwi payload contract; they are synthetic inputs and not physical-meter readings.

TABLE III

Controlled Functional Verification Results

Verification item	Condition	Observed result
Automated service tests	19 tests discovered across User, Device, Ingestion, Usage, Alert, and Insight services; three skipped.	16 executed tests completed with zero failures and zero errors.
Gateway and ingestion	Three analytics events for device 307 plus one isolated alert event for device 308.	Gateway health was UP; all four POST requests returned HTTP 201 (4/4).
Persistence and aggregation	Analytics values: 0.25, 0.50, and 0.75 kWh. Alert value: 1.00 kWh.	Usage API returned 1.50 kWh for device 307 and 1.00 kWh for device 308.
Bill forecast	User 8, D = 3 days, total energy 1.50 kWh.	Current cost 0.18; daily rate 0.50 kWh/day; projected use 15.00 kWh; projected monthly cost 11.80.
Appliance ranking	One isolated device with 1.50 kWh.	Device 307 ranked first with 100.00% contribution and estimated cost 0.18.
Standby heuristic	User 8, D = 3 days, total energy 1.50 kWh.	Baseline 4.17 W; daily 0.10 kWh; monthly 3.00 kWh; monthly cost 0.36; risk LOW.
Threshold alert and delivery	User 9 threshold 0.10 kWh; one 1.00 kWh event.	One detailed alert and one delivery-status record were present. Mailpit captured one email with the 0.10 threshold and 1.00 energy-consumed value.
Output observations	Usage polling at 3 s and alert polling at 5 s.	Analytics output observed within 17.32 s; persisted alert observed within 6.14 s. Both are polling-bounded observations, not latency benchmarks.

B. Interpretation and Evidence Boundaries

The forecast, ranking, and standby results matched the documented deterministic transformations for the controlled 1.50 kWh case. The alert path also demonstrated a direct link from a threshold condition to a stored alert and locally captured email. Kafka is relevant because it separates request acceptance from time-series persistence; InfluxDB is relevant because the prototype queries timestamped device-associated usage independently from MySQL household and device metadata. These are architectural and functional observations, not experimental proof of scalability, reduced latency, fault tolerance, or energy savings.

The 100% acceptance and functional persistence observations are limited to four valid synthetic events in two isolated cases. The 17.32 s analytics observation and 6.14 s alert observation are bounded by client polling and must not be converted into latency or throughput metrics. No benchmark dataset, matched physical-meter series, labelled standby data, baseline implementation, load protocol, resource-utilisation record, or fault-injection protocol was collected. Accordingly, forecast error, anomaly accuracy, precision, recall, F1-score, throughput, resource use, and resilience remain unevaluated.

The optional Ollama integration should be interpreted narrowly. It can produce narrative recommendation wording when available, while a deterministic fallback maintains the interface when it is unavailable. It does not compute bill values, detect anomalies, classify appliance states, or provide a validated predictive inference result.

VI. LIMITATIONS AND FUTURE WORK

The current input is Wokwi-generated synthetic telemetry rather than calibrated smart-meter data. The prototype has no external benchmark dataset, labelled anomaly set, ground-truth bill reference, or empirical baseline. The standby value is a fixed 20% average-load heuristic; the scheduler has no alert-deduplication window; and demonstrated Keycloak/JWT authentication, ownership enforcement, cloud TLS verification, fault injection, and multi-node Kafka resilience validation are absent. The dashboard's illustrative client-side seven-day chart is not treated as backend research evidence.

Future work should integrate calibrated sensing and validated interval-energy conversion, collect consented household telemetry with documented sampling and labels, and evaluate forecast error against matched meter or bill data. It should also replace the fixed standby coefficient with contextual or appliance-level analysis, add authentication and device-provisioning controls, and conduct observability, load, idempotency, retry/DLQ, alert-deduplication, and fault-injection experiments. A trained anomaly-detection model should be selected only after an appropriate labelled dataset and reproducible baseline are available.

VII. CONCLUSION

This paper presents a microservices-based Home Energy Tracker that integrates Wokwi ESP32 telemetry, API Gateway routing, Spring Boot services, Kafka, InfluxDB, MySQL, deterministic energy insights, and threshold-based alerting. The controlled local verification accepted four synthetic events, made the 1.50 kWh analytics and 1.00 kWh alert cases queryable, returned the specified forecast, ranking, and standby outputs, and captured a threshold email through Mailpit. The contribution is a traceable functional integration baseline for future residential-energy experiments. It does not establish physical-meter accuracy, forecasting accuracy, anomaly-detection quality, production scalability, or resilience; those claims require the data and protocols identified in the future-work scope.

REFERENCES

[1] A. Shuaibu, S. U. Haq, B. Almadani, F. Aliyu, and E. Al-Nahari, "Smart Metering Meets AI: Real-Time Appliance Monitoring and Anomaly Detection Over DDS Middleware," IEEE Transactions on Industry Applications, 2026. doi: 10.1109/tia.2025.3608687.

[2] I. Núñez, C. A. Rovetto, E. Cruz, A. Smolarz, D. H. Concepción, and E. Cano, "Design of a microservices-based architecture for residential energy efficiency monitoring," International Journal of Electronics and Telecommunications, 2024. doi: 10.24425/ijet.2024.152511.

[3] W. Agustiarmi et al., "Real-Time IoT-based Energy Power Consumption Monitoring System for Smart Homes," International Journal on Advanced Science, Engineering and Information Technology, 2025. doi: 10.18517/ijaseit.15.5.20918.

[4] M. Islam, N. Jobaida, M. S. Hossain, M. R. Uddin, and M. Hasan, "IoT-Based Smart Energy Monitoring and Energy Demand Response System," 2024 IEEE 3rd International Conference on Electrical Power and Energy Systems (ICEPES), 2024. doi: 10.1109/icepes60647.2024.10653605.

[5] S. S. Shashank, Yogesh, M. Krishnamoorthy, N. Karuppiah, P. Mounica, and R. Abinaya, "Smart Energy Monitoring for Real-Time Power Consumption Tracking and Optimization Using an IoT-Based Cost-Effective System," 2025 11th International Conference on Electrical Energy Systems (ICEES), 2025. doi: 10.1109/icees67011.2025.11212888.

[6] T. B. D. Sousa and R. Rabelo, "Smart Metering Enabled by the Internet of Things for Monitoring Residential Energy Consumption and Power Quality," IEEE Access, 2025. doi: 10.1109/access.2025.3627191.

[7] B. Jyothi et al., "Empowering energy management: anomaly detection in smart meter data for proactive consumption control," Bulletin of Electrical Engineering and Informatics, 2026. doi: 10.11591/eei.v15i3.10957.

[8] K. Kulkarni, "Anomaly Detection in Smart Electric Meters for Detecting Faults and Misuse of Electric Energy Consumption," INTERANTIONAL JOURNAL OF SCIENTIFIC RESEARCH IN ENGINEERING AND MANAGEMENT, 2024. doi: 10.55041/ijsrem34689.

[9] M. AbuArqoub, A. A. Titinchi, and R. Marie, "A Scalable IoT-AI Framework for Real-Time Energy Anomaly Detection in Smart Grid Networks," 2026 2nd International Conference on Computational Intelligence Approaches and Applications (ICCIAA), 2026. doi: 10.1109/icciaa68481.2026.11543846.

[10] Á. Hernández et al., "Detection of Anomalies in Daily Activities Using Data from Smart Meters," Sensors (Basel, Switzerland), 2024. doi: 10.3390/s24020515.

[11] S. Maitra, S. Kundu, and A. Shankar, "A Real-time Anomaly Detection Using Convolutional Autoencoder with Dynamic Threshold," ArXiv, 2024. doi: 10.48550/arxiv.2404.04311.

[12] A. F. S. Veloso, J. Reis, and R. Rabelo, "A Microservices-Based Solution with Hybrid Communication for Energy Management in Smart Grid Environments," Sensors (Basel, Switzerland), 2026. doi: 10.3390/s26051714.

[13] D. K, "Smart Energy Monitoring System," International Journal of Innovative Research in Information Security, 2025. doi: 10.26562/ijiris.2025.v1109.15.

[14] L. L. Motta, W. S. Alvarez, H. R. Carvajal, F. B. Neto, E. D. Lima, and L. F. P. Meloni, "A cloud architecture for home energy management systems: a conceptual model," Energy Informatics, 2025. doi: 10.1186/s42162-025-00599-1.

[15] J. Liu and Y. Hsiao, "Smart plug-based home energy management with IoT and anomaly detection," Journal of the Chinese Institute of Engineers, 2025. doi: 10.1080/02533839.2025.2557233.

[16] K. M. W. Hidayat and M. G. Al-Faris, "IoT-Based Electrical Power Consumption Monitoring System in Households Using ESP32 and PZEM-004T," Brilliance: Research of Artificial Intelligence, 2025. doi: 10.47709/brilliance.v5i2.6368.

[17] S. Sirisumrannukul, T. Intaraumnauy, and N. Piamvilai, "Optimal control of cooling management system for energy conservation in smart home with ANNs-PSO data analytics microservice platform," Heliyon, 2024. doi: 10.1016/j.heliyon.2024.e26937.

[18] M. S. Hossain and M. Hassan, "Smart Energy Meter with Real Time Monitoring System," 2024 13th International Conference on Electrical and Computer Engineering (ICECE), 2024. doi: 10.1109/icece64886.2024.11024970.

[19] D. G. S, T. Gangakhedkar, A. P, A. Ashwija, and C. B. T. Rambapuri, "IOT-Enabled Electrical Consumption Tracking for Intelligent Energy Saving," International Journal of Scientific Research in Engineering and Management, 2025. doi: 10.55041/ijsrem54847.

[20] Y. Zhu and M. Yue, "Design of an IoT-Based Smart City Energy Management System," 2025 4th International Conference on Green Energy and Power Systems (ICGEPS), 2025. doi: 10.1109/icgeps65133.2025.11034388.

[21] S. Susikala, R. K, J. K, S. K, and Y. M, "IoT-Based Smart Electricity Monitoring System with Real-Time Alerts and Remote Control," International Journal of Engineering & Extended Technologies Research, 2026. doi: 10.15662/ijeetr.2026.0802420.

[22] C. S. Sinha, "Smart Meter-Based Power Consumption Detection System: An Integrated Approach Using LSTM Networks and Real-Time Web Interface," International Journal For Multidisciplinary Research, 2025. doi: 10.36948/ijfmr.2025.v07i01.37948.

[23] F. Dinmohammadi, A. M. Farook, and M. Shafiee, "Improving Energy Efficiency in Buildings with an IoT-Based Smart Monitoring System," Energies, 2025. doi: 10.3390/en18051269.

[24] S. Son, S. Ko, H. Lee, S. Lee, and B. Lee, "SIMA: Scalable IoT Microservice Architecture With Users and Things Joint Load Balancing," IEEE Internet of Things Journal, 2024. doi: 10.1109/jiot.2023.3317430.